Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:53 Uhr

ANHANG

E

Java ohne BlueJ

Dieses ganze Buch hindurch haben wir BlueJ benutzt, um unsere Java-Anwendungen zu entwickeln und auszuführen. Dafür gibt es einen guten Grund: BlueJ bietet einige Werkzeuge, die typische Aufgaben bei der Entwicklung erleichtern. Insbesondere ermöglicht BlueJ die direkte Ausführung einzelner Methoden von Klassen und Objekten – das ist nützlich, um schnell ein neues Stück Quelltext zu testen.

Wir unterscheiden bei der Darstellung der Arbeit ohne BlueJ zwei Kategorien: das Ausführen einer Anwendung ohne BlueJ und das Entwickeln ohne BlueJ.

E.1 Java ohne BlueJ ausführen

Anwendungen, die an Benutzer ausgeliefert werden, werden ohne BlueJ gestartet. Sie haben üblicherweise genau einen Einstiegspunkt, an dem die Ausführung beginnt, wenn ein Benutzer die Anwendung startet.

Das konkrete Vorgehen hängt vom jeweiligen Betriebssystem ab. Üblicherweise wird mit einem Doppelklick auf ein Icon gestartet oder indem der Name der Anwendung in eine Kommandozeile eingegeben wird. Das Betriebssystem muss dann wissen, welche Methode welcher Klasse aufgerufen werden soll, um die gesamte Anwendung zu starten.

In Java ist dieses Problem über eine Konvention gelöst: Wenn ein Java-Programm gestartet wird, wird der Name der Klasse als Parameter an das Startkommando übergeben, während der Name der Methode festgelegt ist: **`main`**. Dieser Name wurde willkürlich von den Java-Entwicklern gewählt, steht jedoch fest, d.h., die Methode muss **`main`** heißen. (Der Name dieser Methode geht zurück auf die Startmethode der Programmiersprache C, von der Java ein Großteil der Syntax geerbt hat.)

Betrachten Sie beispielsweise das folgende Kommando, das in eine Kommandozeile, etwa in einem DOS-Fenster unter Windows oder in einer Unix-Shell, eingegeben wird:

```
java Spiel
```

Das Kommando **`java`** startet die virtuelle Maschine von Java. Sie ist Teil des Java Development Kit (JDK), das auf Ihrem System installiert sein muss. **`Spiel`** ist der Name der Klasse, die wir starten wollen.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:53 Uhr

Das Java-System sucht dann in der Klasse **Spiel** nach einer Methode mit der folgenden Signatur:

```
public static void main(String[] args)
```

Die Methode muss öffentlich sein, damit sie von außen aufgerufen werden kann. Sie muss statisch sein, weil zu Anfang noch kein aufrufbares Objekt existiert. Zu Anfang existieren nur Klassen, also können wir nur statische Methoden aufrufen. Diese statische Methode erzeugt dann üblicherweise das erste Objekt. Der Ergebnistyp ist **void**, da die Methode kein Ergebnis zurückliefert.

Der Parameter ist ein Zeichenketten-Array. Dies ermöglicht dem Benutzer, zusätzliche Parameter zu übergeben. In unserem obigen Beispielaufruf ist der Wert des Parameters **args** ein Array mit der Länge null sein. In der Kommandozeile können aber auch Parameter angegeben werden, etwa folgendermaßen:

```
java Spiel 2 Fred
```

Jedes Wort in der Kommandozeile nach dem Klassennamen wird als eine separate Zeichenkette aufgefasst und der Methode **main** als ein String-Element im String-Array übergeben. In diesem Fall würde das Array **args** zwei Elemente enthalten, die beiden Strings **"2"** und **"Fred"**. Kommandozeilenparameter werden mit Java nur sehr selten verwendet.

Der Rumpf der Methode **main** kann theoretisch beliebige Anweisungen enthalten. Es ist jedoch guter Stil, wenn diese Methode möglichst klein gehalten wird. Insbesondere sollte hier keine Anwendungslogik implementiert sein.

Typischerweise sollte die Methode **main** genau das Gleiche tun, das Sie auch zum interaktiven Starten in BlueJ tun würden. Wenn Sie beispielsweise ein Objekt der Klasse **Spiel** erzeugen würden, um daran die Methode **starten** aufzurufen, dann sollten Sie folgende **main**-Methode in Ihrer Klasse **Spiel** einfügen:

```
public static void main(String[] args)
{
    Spiel spiel = new Spiel();
    spiel.starten();
}
```

Auf diese Weise ahmt die Methode **main** ein interaktives Starten nach.

Jedes Java-Projekt liegt üblicherweise in einem eigenen Verzeichnis. Alle Klassen des Projekts sind in diesem Verzeichnis abgelegt. Wenn Sie das Kommando zum Starten von Java aufrufen und Ihre Anwendung ausführen wollen, dann sollten Sie sicherstellen, dass das Projektverzeichnis das aktuelle Verzeichnis in Ihrer Kommandoumgebung ist. Die Klassen können dann gefunden werden.

Wenn die angegebene Klasse nicht gefunden werden kann, gibt die virtuelle Maschine von Java eine Fehlermeldung der folgenden Art aus:

```
Could not find or load main class Spiel
```

Wenn Sie eine Meldung wie diese bekommen, dann stellen Sie sicher, dass Sie den Klassennamen korrekt eingegeben haben und das aktuelle Verzeichnis diese Klasse auch wirklich enthält. Die Klasse wird in einer Datei mit der Endung *.class*

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:53 Uhr

gespeichert. Der Code für die Klasse **Spiel** beispielsweise ist in einer Datei mit dem Namen *Spiel.class* gespeichert.

Wenn diese Klasse gefunden wird, aber keine **main**-Methode enthält (oder eine **main**-Methode mit der falschen Signatur), werden Sie eine Meldung erhalten, die ungefähr so aussieht:

```
Main method not found in class Spiel, please define the main method as:
public static void main(String[] args)
```

In diesem Fall sollten Sie sicherstellen, dass Ihre Klasse eine Methode **main** mit der richtigen Signatur definiert.

E.2 Konsolenanwendungen und die Problematik der Umlaute

Von den GUI-Anwendungen aus Kapitel 13 einmal abgesehen, haben wir uns in den Beispielprogrammen dieses Buches zumeist der Methode **System.out.println()** bedient, um Ergebnisse und Meldungen auf die Konsole auszugeben. Solange Sie in BlueJ arbeiten und die Programme oder Methoden von BlueJ aus starten, schickt **System.out.println()** sämtliche Ausgaben an die BlueJ-Konsole. Sobald Sie Ihre Konsolenanwendungen aber ohne BlueJ starten, gehen alle Ihre **System.out**-Ausgaben an die Konsole, von der aus das Programm – wie im vorangehenden Abschnitt beschrieben – gestartet wurde. Für deutschsprachige Konsolenanwendungen unter Windows hat dies in der Regel einen unangenehmen Nebeneffekt: Die deutschen Umlaute werden nicht mehr korrekt angezeigt.

Wenn Sie Zeichenketten via **System.out** auf die Windows-Konsole ausgeben, werden die 16-Bit-Codes der einzelnen Zeichen auf je 8 Bit zurechtgestutzt, ohne dass dabei allerdings eine korrekte Umcodierung in den 8-Bit-OEM-Zeichensatz der Konsole stattfinden würde. Das traurige Ergebnis: Die deutschen Umlaute sowie verschiedene weitere Umlaute und Sonderzeichen, die die Konsole prinzipiell anzeigen könnte, gehen verloren.

Um die Umlaute dennoch korrekt auszugeben, müssen Sie

- entweder die Zeichenkodierung des **System.out**-Objekts anpassen
- oder auf das in Java 6 neu eingeführte **Console**-Objekt zurückgreifen.

E.2.1 Zeichenkodierung für System.out anpassen

Wie Sie aus Kapitel 2 bereits wissen, verbirgt sich hinter **System.out** ein **PrintStream**-Objekt. Dieses Objekt ist mit der Konsole als Ausgabegerät verbunden und verwendet die Standard-Zeichenkodierung, welche von der Konfiguration des zugrunde liegenden Betriebssystems abhängt. Wenn Sie eigene **PrintStream**-Objekte erzeugen, können Sie die Zeichenkodierung frei wählen (soweit verfügbar).

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:53 Uhr

```
public static void main(String[] args)
{
    java.io.PrintStream out;

    try {
        System.setOut(new java.io.PrintStream(System.out, true, "Cp850"));
    }
    catch (java.io.UnsupportedEncodingException e) {
    }

    Spiel spiel new Spiel();
    spiel.starten();
}
```

Dem **PrintStream**-Konstruktor werden hier drei Argumente übergeben:

- ein **OutputStream**-Objekt, das das Ziel der Ausgabe vorgibt (hier die Konsole)
- **true**, damit die Ausgaben sofort ausgeführt werden (andernfalls werden die Ausgaben gepuffert, und Sie müssen die Methode **flush()** aufrufen)
- den Namen der gewünschten Zeichenkodierung (hier **"Cp850"** für die DOS-Codepage 850)

Danach wird das neu erzeugte **PrintStream**-Objekt mittels der **setOut()**-Methode als neues Standardausgabegerät registriert.

E.2.2 Das Console-Objekt

Die Klasse **Console** instanziieren Sie nicht selbst. Wenn Ihr Java-Code im Kontext einer Java-Virtual-Machine-Instanz ausgeführt wird, die mit einem Konsolenfenster verbunden ist, erzeugt die JVM automatisch intern ein **Console**-Objekt, welches das Konsolenfenster repräsentiert. Über die statische **Console**-Methode **console()** können Sie sich eine Referenz auf dieses Objekt zurückliefern lassen.

```
java.io.Console cons = System.console();
```

Wenn die Java Virtual Machine kein beigeordnetes Konsolenfenster ermitteln kann, liefert **System.console()** eine **null**-Referenz zurück. Dies ist in der Regel dann der Fall, wenn die Anwendung nicht aus einem Konsolenfenster heraus gestartet wurde (also beispielsweise auch bei der Ausführung innerhalb von BlueJ). Zur Sicherheit sollten Sie den Rückgabewert von **System.console()** daher immer überprüfen.

Text auf die Konsole ausgeben

Alle Ausgaben, die Sie über das von **System.console()** zurückgelieferte **Console**-Objekt vornehmen, werden automatisch an die Zeichenkodierung der Konsole angepasst.

```
java.io.Console cons = System.console();

if (cons != null) {
    cons.printf("\n");
    cons.printf(" Ausgabe der Umlaute mit Console \n");
    cons.printf(" ä, ö, ü, ß \n");
}
```


Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:53 Uhr

Beachten Sie, dass die Ausgabemethode der Klasse **Console** nicht **println()**, sondern **printf()** heißt. Die Methode kann analog zur **println()**-Methode von **System.out** verwendet werden, gestattet aber auch die Ausgabe mit Formatierungsstrings und Platzhaltern, die Umsteigern von C oder C++ vermutlich bekannt sein wird. So erzeugt der Code

```
java.io.Console cons = System.console();
String dvd = "Solaris";
double preis = 12,95;

if (cons != null) {
    cons.printf("Die %s-DVD kostet %4.2f", dvd, preis);
}
```

die Ausgabe

```
Die Solaris-DVD kostet 12,95
```

Von der Tastatur lesen

Über das vordefinierte **Console**-Objekt können Sie auch Tastatureingaben einlesen. Am einfachsten geht dies mit der Methode **readLine()**, die eine einzelne Eingabezeile einliest und als **String**-Objekt zurückliefert. Numerische Eingaben können Sie zeilenweise einlesen und mit den **Parse**-Methoden der verschiedenen Wrapper-Klassen in die entsprechenden Werte umwandeln.

```
java.io.Console cons = System.console();
String name = "";
int alter = 0;

if (cons != null) {
    cons.printf(" Geben Sie Ihren Namen ein: ");
    name = cons.readLine();
    cons.printf(" Geben Sie Ihr Alter ein: ");
    alter = Integer.parseInt( cons.readLine() );
    cons.printf (" %s (Alter: %d) \n", name, alter);
}
```

Wenn Sie Passwörter oder andere sensible Daten einlesen möchten, die für Umstehende nicht lesbar sein sollen, rufen Sie statt **readLine()** die Methode **readPassword()** auf.

E.3 Ausführbare jar-Dateien erzeugen

Java-Projekte werden üblicherweise als eine Sammlung von Dateien in einem Verzeichnis abgelegt. Wir werden unten kurz die beteiligten Dateien beschreiben.

Um eine Anwendung an andere weitergeben zu können, ist es oft einfacher, wenn alle Teile in einer einzelnen Datei zusammengefasst sind. Der Mechanismus von Java für diesen Zweck ist das *Java Archive Format* (*jar*). Alle Dateien einer Anwendung können dabei in einer einzelnen Datei zusammengefasst und dennoch ausführbar sein. (Wenn Sie bereits das Komprimierungsformat *zip* kennen: Es ist das gleiche Format. *jar*-Dateien können mit Zip-Programmen geöffnet werden und umgekehrt.)

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:53 Uhr

Um eine *jar*-Datei ausführbar zu machen, muss festgelegt werden, welche die Startklasse ist. (Sie erinnern sich: Die Methode zum Starten heißt immer **main**, aber wir müssen angeben, in welcher Startklasse diese Methode liegt.) Dazu wird in einer *jar*-Datei eine Textdatei abgelegt (die so genannte *Manifest-Datei*), die diese Information enthält. BlueJ bietet hierbei Unterstützung an.

Um eine ausführbare *jar*-Datei in BlueJ zu erstellen, können Sie im Menü PROJEKT den Eintrag ALS JAR-ARCHIV SPEICHERN aufrufen und im folgenden Dialog die Klasse auswählen, die die **main**-Methode enthält. (Sie müssen die **main**-Methode wie oben beschrieben implementieren.)

Für weitere Fragen zu dieser Operation können Sie das BlueJ-Tutorial lesen, das Sie im Menü HILFE über den Eintrag BLUEJ-TUTORIAL oder auf den BlueJ-Webseiten finden. Sobald die ausführbare *jar*-Datei erzeugt wurde, kann sie mit einem Doppelklick gestartet werden. Dazu muss auf dem Rechner, der die *jar*-Datei ausführen soll, ein JDK (ein Java Development Kit) oder ein JRE (Java Runtime Environment) installiert und mit *jar*-Dateien assoziiert sein.

E.4 Entwickeln ohne BlueJ

Wenn Sie Ihre Programme nicht nur ohne BlueJ ausführen, sondern auch ohne BlueJ entwickeln wollen, dann müssen Sie Ihre Klassen editieren und übersetzen. Der Quelltext einer Klasse wird in einer Datei mit der Endung *.java* gespeichert. Beispielsweise ist die Klasse **Spiel** in einer Datei mit dem Namen *Spiel.java* gespeichert. Quelltextdateien können mit jedem Texteditor bearbeitet werden. Es gibt sehr viele frei verfügbare oder kostengünstige Texteditoren. Einige, wie *Notepad* oder *Wordpad*, werden mit Windows ausgeliefert; aber wenn Sie den Editor für mehr als nur für einen schnellen Test benutzen wollen, dann werden Sie sich schnell etwas Besseres besorgen wollen. Seien Sie jedoch vorsichtig mit Textverarbeitungsprogrammen; diese speichern üblicherweise nicht im einfachen Textformat, und das Java-System kann die Dateien dann nicht lesen.

Die Quelltextdateien können dann in der Kommandozeile übersetzt werden, indem Sie den Compiler aufrufen, der mit dem JDK ausgeliefert wird. Er heißt **javac**. Um eine Quelltextdatei *Spiel.java* zu übersetzen, benutzen Sie das Kommando

```
javac Spiel.java
```

Dieses Kommando wird die Klasse **Spiel** sowie alle Klassen, die von dieser Klasse abhängen, übersetzen und dann eine Datei *Spiel.class* anlegen. Diese Datei enthält den Code, der von der virtuellen Maschine für Java ausgeführt werden kann. Um sie auszuführen, benutzen Sie das Kommando

```
java Spiel
```

Beachten Sie, dass bei diesem Aufruf die Endung *.class* nicht mit angegeben ist.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:53 Uhr

ANHANG

F

Benutzung des Debuggers

Der Debugger in BlueJ bietet eine Reihe von absichtlich vereinfachten, aber sehr nützlichen Funktionen zum Debuggen von Programmen und zum leichteren Verstehen des Laufzeitverhaltens von Programmen.

Das Debugger-Fenster wird hervorgeholt, wenn der Eintrag DEBUGGER ANZEIGEN aus dem Menü ANSICHT ausgewählt wird oder wenn über dem Aktivitätsbalken mit der rechten Maus geklickt wird und der Eintrag ZEIGE DEBUGGER aus dem Kontextmenü ausgewählt wird. Abbildung F.1 zeigt das Debugger-Fenster.

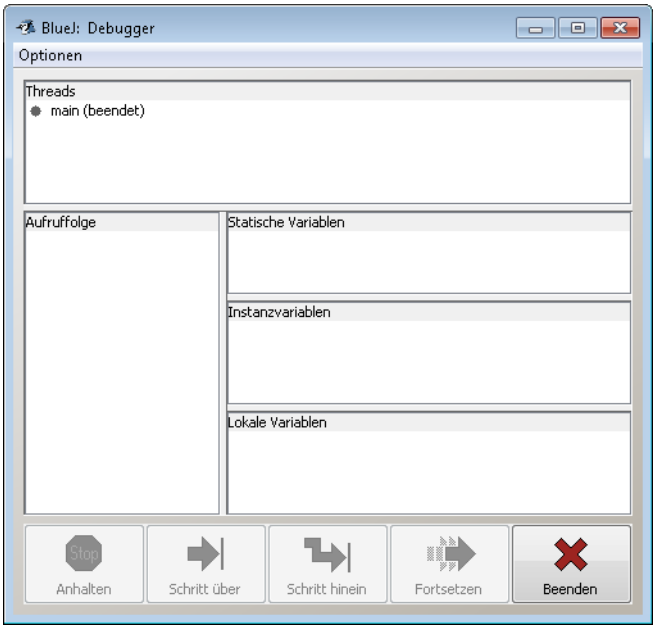

Abbildung F.1
Das Debugger-Fenster von BlueJ.

Das Debugger-Fenster enthält fünf Anzeigebereiche und fünf Kontrollknöpfe. Die Bereiche und die Knöpfe werden nur aktiviert, wenn ein Programm einen Haltepunkt erreicht oder aus einem anderen Grund anhält. Die folgenden Abschnitte beschreiben, wie Haltepunkte gesetzt werden, wie die Programmausführung kontrolliert wird und welchen Zweck die verschiedenen Anzeigebereiche haben.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:53 Uhr